温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3101 W03 Foundations of Business Analytics

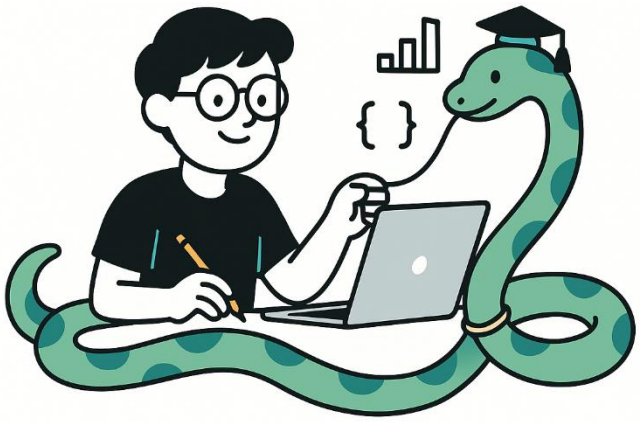
Jupyter Notebook, Python Basics

Dawei Chen

Week 2-1, 09 Sep 2025

How Much Python Do You Need for Beginner Data Analysis?

- Not much – but basics are required
  - Enough to read and understand simple Python code
  - The more Python you know, the better
- Not a Python programming course
  - Focus is on practical use for data analysis
- My experience
  - Not a CS major, not an expert in Python
  - Still able to use Python effectively for data analysis
- Best way to learn
  - learn by practice and doing
  - You do not need to know everything before starting
  - Problems will appear ➡ solve them step by step ➡ you learn



Generated by ChatGPT5

Outline

- Example Data Analysis Project
- Introduction to Jupyter Notebook
  - What, why, how
- Python Basics Review
  - Key concepts
  - Control flows
  - Functions
  - Classes and objects
  - Modules, packages and libraries
  - Errors and exceptions
- Summary

Generated by ChatGPT5

An Example: EDA To Prediction(DieTanic)

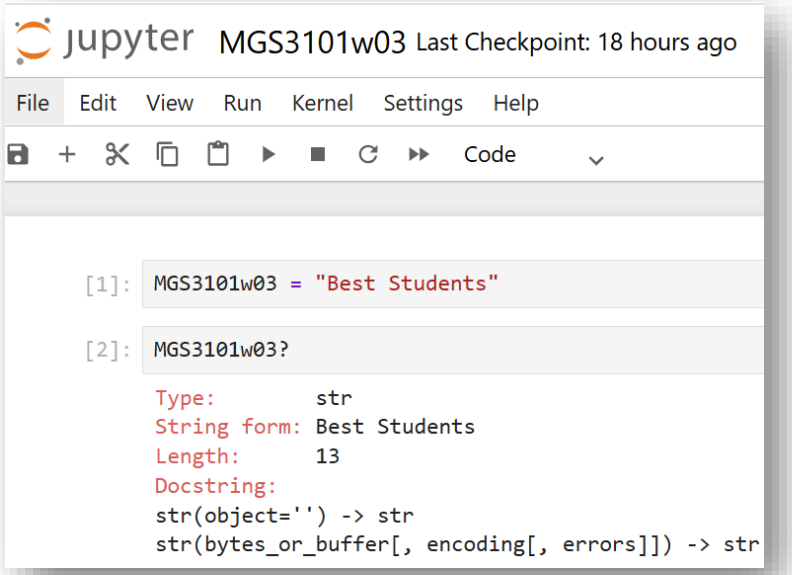
- Structure and Layout
- Cell Types
  - Code cells: Python codes
  - Text (Markdown) cells: titles, notes, equation
- Workflow
  1) Setup: import libraries, set options/paths, set random seed
  2) Load data: read files, glance
  3) Exploratory analysis: summary stat, missing values, outliers, visuals, correlations, etc.
  4) Data cleaning and feature engineering: type/format conversion, imputation, outliner handling, scaling/standardization, new features, etc.
  5) Modeling & evaluation: selection, training, validation, sensitive analysis, iteration & improvement
  6) Save & report: export figures/tables/models, write takeaways

Jupyter Notebook – What & Why

- Definition
  - A open-source web application that allows you to create and share documents that contain a mix of live code, equations, visualizations and explanatory text.
- Advantages
  - **Interactive & incremental:** run one cell at a time, instant feedback and easy debugging
  - **Literate workflow:** combine code, Markdown text, and LaTeX to explain each step
  - **Visual by default:** inline table, chart, HTML for quick exploration
  - **Shareable and reproducible:** one .ipynb file, export to HTML/PDF, view on GitHub, etc.
  - **Language-flexible:** multiple kernels (Python, R, Julia) under the same interface

Jupyter Notebook - How

- Installation and launch
- Files
  - Navigate to your folder -> New, Python 3 (ipykernel) -> rename -> save (.ipynb)
- Menu bar
  - **File:** new / open / save / save as / rename …
  - **Edit:** cell operations, find/replace, shortcuts
  - **Run:** run selected cell (Shift+Enter), run all / above / below
  - **Kernel:** separate processes started by the server that run your code in different programming languages and environments
  - **Help:** keyboard shortcuts, markdown reference



Python Basics – Key Concepts

Variables

- Create
- Naming rules
- Naming conventions

Assignment is not equal to

```
[4]: a = [1, 2, 3]
[5]: b = a
[6]: a
[6]: [1, 2, 3]
[7]: b
[7]: [1, 2, 3]
```

```
[8]: a = [1, 2, 3, 4]
[9]: b
[9]: [1, 2, 3]
```

```
[8]: a.append(4)
[9]: b
[9]: [1, 2, 3, 4]
```

MGS3101w03 = "Best Students"

Built-in Data Types

- Text: str — 'WKU is the best'
- Numeric: int, float, complex — 666, 66.6, 6+6j
- Sequence: list, tuple, range — [1, 2, 3], (1, 2, 3), range(3)
- Mapping: dict — {MGS3010: w03}
- Set: set, fronzenset — {1, 2, 3}
- Boolean: bool — True, False
- Binary: bytes, beyearray, memoryview
- None: NoneType — None

Other Data Types?

- Different modules define various data types

Python Basics – Operators

- Arithmetic
  - +, -, *, /, %, **, //
- Assignment
  - =, +=, -=, *=, /=, %=, //=, **=, &=, |=, ^=, >>=, <<=, :=
- Comparison
  - ==, !=, >, <, >=, <=
- Logical
  - and, or, not
- Identity
  - is, is not
- Membership
  - in, not in
- Bitwise
  - &, |, ^, ~, <<, >>

Operator Precedence

They are useful for filtering DataFrame by conditions during analysis.

Python Basics – Control Flows

- if, elif, and else

```
if x < 0:
    print("It's negative")
elif x == 0:
    print("Equal to zero")
elif 0 < x < 5:
    print("Positive but smaller than 5")
else:
    print("Positive and larger than or equal to 5")
```

- for loops

```
sequence = [1, 2, None, 4, None, 5]
total = 0
for value in sequence:
    if value is None:
        continue
    total += value
```

- while loops

```
x = 256
total = 0
while x > 0:
    if total > 500:
        break
    total += x
    x = x // 2
```

- range

```
In [135]: range(10)
Out[135]: range(0, 10)

In [136]: list(range(10))
Out[136]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Python Basics – Functions

A function is a block of code which only runs when it is called.

Name of function

```
def my_function(x, y):
    return x + y
```

Output

Parameters/Arguments

- Arbitrary, *arg
- Keyword
- Arbitrary keyword, **kwargs
- Positional-only
- Keyword-only
- etc.

What should care when you call a function?

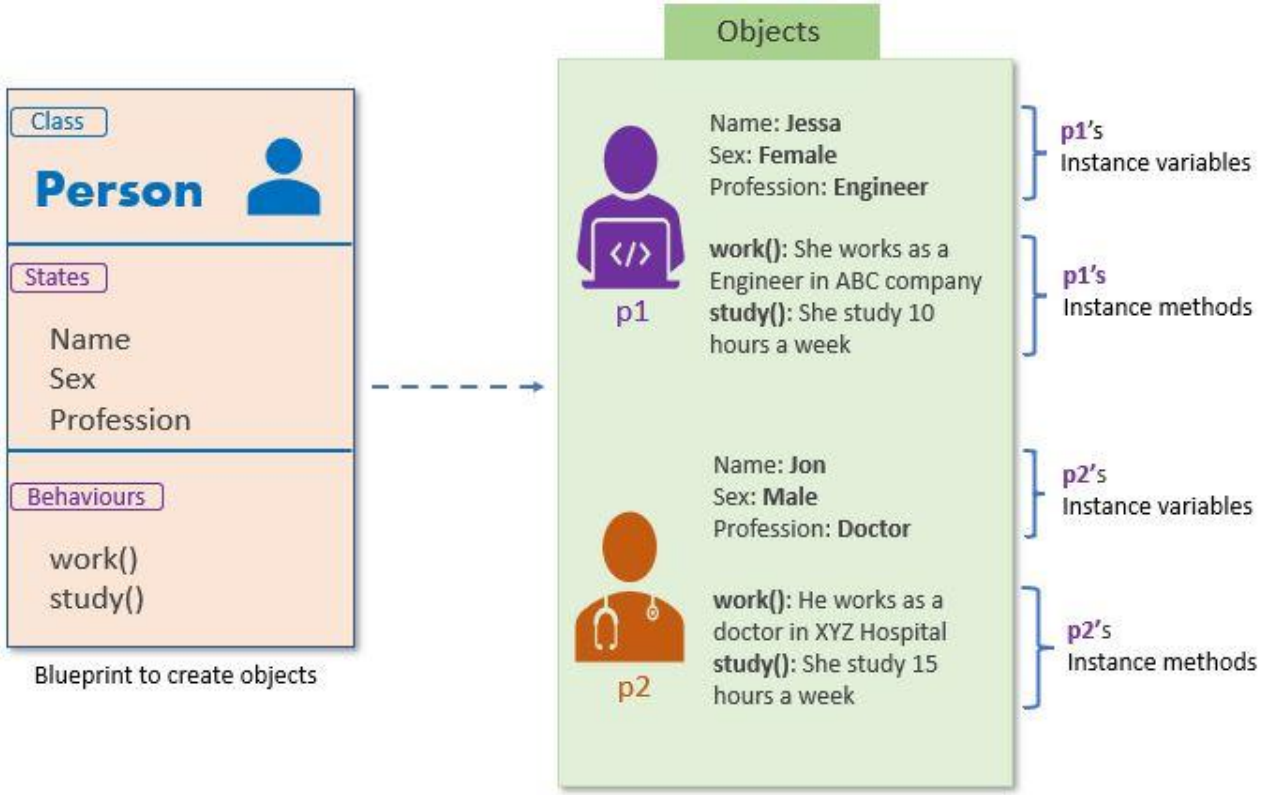
- Purpose
- Arguments
  - which one?
  - required vs optional
  - dtype of input
  - Order
  - How it handles NaN, errors
- Output: dtype

How to call a function?

- Built-in functions (global): len(obj)
- Object method (on the object itself): data.sum()
- Module/package functions: pd.read_csv('file.csv')

An Example

Python Basics – Classes & Objects



https://pynative.com/python-classes-and-objects/

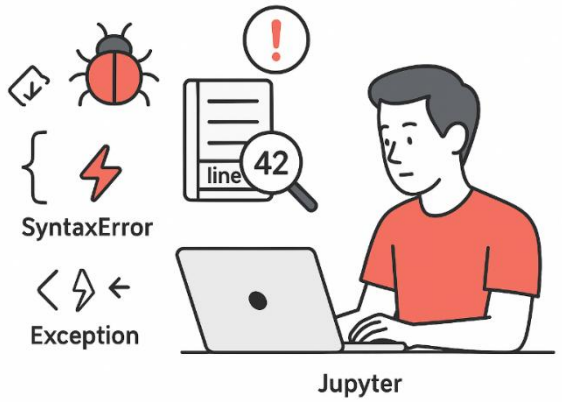
- The class is a user-defined data structure that binds the data members and methods into a single unit.
- An object is an instance of a class.
- Everything is an object.
- Each object has its own, identity, attributes (state) and methods (behavior).

Python Basics – Modules & Packages & Libraries

- Why they exist
  - Reuse code across sessions (no re-typing)
  - Organize longer programs into manageable pieces
  - Share handy functions across projects
- What they are
  - Modules: a .py file containing functions/variables/classes
  - Package: a folder that groups related modules
  - Library: a collection of modules/packages (e.g., Python's standard library)
- How to use them
  - Import a module: `import math`
  - Import a name: `from math import sqrt`
  - Use an alias: `import pandas as pd`
  - Get quick help in Jupyter: `math.sqrt?`

Python Basics – Errors and Exceptions

- What can go wrong
  - **SyntaxError:** Python can't parse your code (typo/format)
  - **Exceptions (runtime):** Code parsed, but fails while running
- Read the traceback
  - Last line = **exception type** + **message**
  - Use **file** + **line number**, start fixing at the highest frame in your code
  - Get help from AI tools, online community (e.g., Stack Overflow), Google, etc.
- Common exceptions in analysis
  - **NameError:** variable not defined
  - **TypeError:** wrong type /wrong argument
  - **ValueError:** right type, bad value
  - **KeyError:** missing column/key
  - etc.



Generated by ChatGPT5

Revisit DieTanic
Understand Python Code

Summary

- Python self-learning resources
  - W3School / 菜鸟教程 / The Python Tutorial
- Jupyter notebook contains a mix of live code, equations, visualizations and explanatory text for better interaction, exploration, visualization, collaboration and reproduction.
- Python basics
  - data types, operators, control flows, functions, classes, modules & packages & libraries

elements → *a line of code* → *blocks of code* → *.py files and folders* → Complexity

- Next class (Thu): NumPy & Pandas